

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Lab 5 - ARP Poisoning Forensics

Lab Title:	ARP Poisoning Forensics
Student Name:	Yusuf Oluwaseun Dada
Student ID:	2025/FWSD/11339
Course Code:	SBT-DF203
Course Name:	Basic Networking Skills for Digital Forensics

Instructor: Mr. Aminu Idris, AMCPN

Date: 17/09/2026

1. Lab Scenario

A user reports that the default gateway intermittently resolves to the wrong MAC address. I will need to first document normal ARP resolution, then analyze a known ARP-poisoning capture. In an instructor-approved host-only network, which i may reproduce a short controlled poisoning event, capture the evidence, restore the ARP mappings and explain how the attack is detected.

2. Learning Outcomes

1. Explain how ARP maps IPv4 addresses to MAC addresses on a LAN.
2. Inspect ARP caches on Linux and Windows systems.
3. Identify ARP requests, legitimate replies and unsolicited or conflicting replies.
4. Detect one IP address being associated with multiple MAC addresses.
5. Capture and document a bounded ARP-poisoning simulation in a host-only network.
6. Restore network state and recommend preventive controls.

3. Required Tools and Environment

Item	Recommended Tool	Purpose
Virtual machines	Kali attacker/analyst plus Linux or Windows victim	Create an isolated LAN scenario.
Network mode	Host-only/internal network	Prevent access to production networks.
Capture tools	Wireshark and tshark	Capture ARP packets.
Packet library	python3-scapy	Run authorized training scripts.
System tools	ip neigh, arp, ipconfig /all	Inspect ARP tables and interfaces.
Evidence	arp.pcap or student capture	Analyze known poisoning indicators.

4. Set Up the Lab Folder and Evidence

```
(kali㉿kali)-[~/Desktop]
└─$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5
pwd
find . -maxdepth 1 -type d -print
/home/kali/SBT-DF203-Lab5
.
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ wget -O evidence/arp.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
--2026-09-16 18:48:43-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.108.133, 185.199.111.133, 185.199.109.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.108.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap      100%[=====>] 1.05K --KB/s in 0s

2026-09-16 18:48:44 (12.9 MB/s) - 'evidence/arp.pcap' saved [1072/1072]
```

```
(kali㉿kali)-[~/SBT-DF203-Lab5]
└─$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap
sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
ip -br address | tee reports/interfaces.txt
ip route | tee reports/routes.txt
ip neigh show | tee reports/arp_table_initial.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.229.128/24 fe80::c48e:ee4c:6374:462d/64
docker0 DOWN 172.17.0.1/16
default via 192.168.229.2 dev eth0 proto dhcp src 192.168.229.128 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.229.0/24 dev eth0 proto kernel scope link src 192.168.229.128 metric 100
```

wget downloads the instructor-supplied reference ARP-poisoning capture into evidence/ treat it as original and not to edit it directly. cp makes my working copy; sha256sum immediately fingerprints both matches. ip -br address, ip route, and ip neigh show record your interface addresses, default gateway, and current ARP cache

5. Chain-of-Custody Worksheet

Field	Your Entry
Case/lab identifier	SBT-DF203-Lab5
Trainee name	Yusuf Oluwaseun Dada
Date and time started	15/09/2026 4:30pm
Evidence file name(s)	arp.pcap
Source or generation method	Downloaded reference capture / self-generated
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Working-copy SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e

6. Part A - Observe Normal ARP Resolution

1. Identify the victim IP, gateway IP, and your interface name (from reports/interfaces.txt and reports/routes.txt).
2. Clear only the specific training neighbor entry where permitted — never flush unrelated entries on a shared machine.
3. Start an ARP-only capture, ping the gateway once, and stop the capture.
4. Identify the broadcast request and the unicast reply in the capture.

```
└─$ IFACE=eth0
GATEWAY_IP=$(ip route | awk '/default/ {print $3; exit}')
sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
sudo tshark -i "$IFACE" -f "arp" -a duration:20 -w /normal_arp.pcapng &
sleep 2
ping -c 1 "$GATEWAY_IP"
wait
ip neigh show | tee reports/arp_table_after_ping.txt
[1] 21175
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
PING 192.168.229.2 (192.168.229.2) 56(84) bytes of data.
64 bytes from 192.168.229.2: icmp_seq=1 ttl=128 time=0.618 ms

— 192.168.229.2 ping statistics —
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.618/0.618/0.618/0.000 ms
9
[1] + done          sudo tshark -i "$IFACE" -f "arp" -a duration:20 -w /normal_arp.pcapng
192.168.229.2 dev eth0 lladdr 00:50:56:f1:f5:3b REACHABLE

(kali@kali)-[~/SBT-DF203-Lab5]
└─$ sudo mv /normal_arp.pcapng evidence/normal_arp.pcapng
```

`$(ip route | awk '/default/ {print $3; exit}')` automatically extracts the default gateway's IP address from the routing table, so no need to hardcode it. `ip neigh flush` removes only that one gateway entry from the ARP cache, forcing a fresh ARP resolution to happen (rather than reusing a cached mapping) when ping next. `tshark -f "arp"` captures only ARP packets (broadcast requests and unicast replies), capped at 20

7. Part B - Analyze ARP Request and Reply Fields

```
(kali@kali)~/SBT-DF203-Lab5
$ sudo tshark -r evidence/normal_arp.pcapng -Y "arp" -T fields \
  -e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/normal_arp_fields.tsv
Running as user "root" and group "root". This could be dangerous.
1 2026-09-16T18:58:44.424939886-0400 00:0c:29:e6:98:7a ff:ff:ff:ff:ff:ff 1 192.168.229.128 00:0c:29:e6:98:7a 1
92.168.229.2 00:00:00:00:00:00
2 2026-09-16T18:58:44.425188473-0400 00:50:56:f1:f5:3b 00:0c:29:e6:98:7a 2 192.168.229.2 00:50:56:f1:f5:3b 1
92.168.229.128 00:0c:29:e6:98:7a
3 2026-09-16T18:58:56.315236461-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
4 2026-09-16T18:58:57.204039645-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
5 2026-09-16T18:58:58.204172825-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
6 2026-09-16T18:58:59.322620762-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
7 2026-09-16T18:59:00.196315521-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
8 2026-09-16T18:59:01.196041397-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
9 2026-09-16T18:59:02.328461190-0400 00:50:56:c0:00:08 ff:ff:ff:ff:ff:ff 1 192.168.229.1 00:50:56:c0:00:08 1
92.168.229.2 00:00:00:00:00:00
```

It extracts every ARP-relevant field from your baseline capture: the Ethernet source/destination, the ARP opcode (1 = request, 2 = reply), and the sender/target protocol (IP) and hardware (MAC) addresses carried inside the ARP packet itself.

Field	Normal Request	Normal Reply
Ethernet destination	ff:ff:ff:ff:ff:ff	00:0c:29:69:98:7a
ARP opcode	1	2
Sender protocol address	192.168.229.128 (Requester IP)	192.168.229.2 (Target IP)
Sender hardware address	00:0c:29:69:98:7a-Requester MAC	00:50:56:f1:f5:3b-Target MAC
Target protocol address	192.168.229.2-Target IP	192.168.229.128-Requester IP
Forensic interpretation	Who has target 192.168.229.2?	192.168.229.2 is at 00:50:56:f1:f5:3b Target IP is at target MAC.

8. Part C - Analyze the Supplied Poisoning Capture

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ PCAP=working/arp_working.pcap
# Inventory all ARP claims
tshark -r "$PCAP" -Y "arp.opcode==2" -T fields \
  -e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac -e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
  | tee reports/arp_replies.tsv
4      1678591364.904123642      00:50:56:86:02:65      00:50:56:86:cb:fc      136.160.215.194 00:50:56:86:02:65      136.160.215.15 00:
50:56:86:cb:fc
6      1678591369.943043799      00:50:56:86:cb:fc      00:50:56:86:02:65      136.160.215.15 00:50:56:86:cb:fc      136.160.215.194 00:
50:56:86:02:65
```

arp.opcode==2 isolates only ARP replies (as opposed to requests), replies are what actually update a victim's ARP cache, so they're where poisoning claims live. Extracting the sender's claimed IP-to-MAC mapping for every reply in the capture gives a complete inventory of every identity claim made during the session, in time order.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ # Summarize IP-to-MAC claims
tshark -r "$PCAP" -Y "arp.opcode==2" -T fields -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort | uniq -c | sort -nr | tee reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
```

Pipes every (claimed IP, claimed MAC) pair through sort | uniq -c, counting how many times each distinct pairing appears, and sort -nr ranks the most frequent claims first. This is the single most important output in the lab: that is direct, textbook evidence of ARP poisoning, a legitimate host only ever advertises one MAC for its own IP.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ # Look for gratuitous or unsolicited-looking replies
tshark -r "$PCAP" -Y "arp.opcode==2 && eth.dst!=ff:ff:ff:ff:ff:ff" -T fields \
  -e frame.number -e frame.time -e arp.src.proto_ipv4 -e arp.src.hw_mac -e eth.dst \
  | tee reports/unicast_arp_replies.tsv
4      2023-03-11T22:22:44.904123642-0500      136.160.215.194 00:50:56:86:02:65      00:50:56:86:cb:fc
6      2023-03-11T22:22:49.943043799-0500      136.160.215.15 00:50:56:86:cb:fc      00:50:56:86:02:65
```

Normal ARP replies are unicasts, but a poisoning attack often sends gratuitous ARP replies, unsolicited announcements pushed out without a matching prior request, sometimes broadcast, sometimes sent directly to a victim who never asked. This filter excludes broadcast-destination traffic and lists direct replies so you can cross-reference their timing against request traffic: a reply with no corresponding recent request in reports/arp_replies.tsv is suspicious.

Question	Your Finding
Which IP address shows multiple claimed MACs?	None found in the ARP reply data — each of the two observed replies (136.160.215.194 → 02:65, 136.160.215.15 → cb:fc) is self-consistent and internally correct
What are the conflicting MAC addresses?	No conflicting MAC-to-IP claims present in the reply set
How many unsolicited/gratuitous replies were found?	0 confirmed by opcode==2 mismatch check; recommend checking request/reply pairing and full opcode==1 traffic before concluding
Which host (attacker MAC) is issuing the	00:50:56:86:cb:fc

Question	Your Finding
false claims?	